In [ ]:
```
# Input functions continuation
```

In [2]:
```
exp=input('Enter a value')
print(exp)
```

```
2+3-5
```

In [3]:
```
exp=int(input('Enter a value')) #expressions work only with evaluate called eval
print(exp)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[3], line 1
----> 1 exp=int(input(               ))
      2 print(exp)

ValueError: invalid literal for int() with base 10: '2+3-4'
```

In [4]:
```
exp=eval(input('Enter a value')) # thus its working
print(exp)
```

```
-1
```

List Data Structures

it always defines with square braces

In [22]:
```
l = []
l
```

Out[22]:
```
[]
```

In [23]:
```
type(l)
```

Out[23]:
```
list
```

In [24]:
```
type(list(l))
```

Out[24]:
```
list
```

In [25]:
```
l.append(20)     #adding the value at end of list is done using append
l.append(50)
l.append(70)     # you can append many number of times
l
```

Out[25]:
```
[20, 50, 70]
```

In [26]:
```
len(l)
```

Out[26]: 3

In [65]:
```
l.append(20)   #it allows duplicates as well
l
```

Out[65]: [20, 50, 70, 20, 20]

In [98]:
```
l1 = [20, 3.4, 3+2j, True,'hello', [1,4,5]]  #heterogenoue i.e all data types pf va
l1
```

Out[98]: [20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]

In [99]:
```
len(l1)
```

Out[99]: 6

In [68]:
```
len(l)
```

Out[68]: 5

In [69]:
```
print(l)
print(l1)
```

```
[20, 50, 70, 20, 20]
[20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]
```

In [70]:
```
print(id(l))
print(id(l1))   #memory location address is called as ID
```

```
1924782990592
1924786855552
```

In [71]:
```
l2 = l.copy()
l2
```

Out[71]: [20, 50, 70, 20, 20]

In [72]:
```
len(l2)
```

Out[72]: 5

In [73]:
```
l2.clear()  # it clears the l2 data and length becomes zero
```

In [74]:
```
len(l2)
```

Out[74]: 0

In [75]:
```
del l2
#the del will delete the l2 since it doesnt have any values in it
#where it automaticlly goes to garbage collector
```

In [40]:
```
l2
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[40], line 1
----> 1 l2

NameError: name 'l2' is not defined
```

In [76]:
```
l.count(20)  # this is used to count how many times 20 is available in l
```

Out[76]:
```
3
```

In [77]:
```
l.count(100)
```

Out[77]:
```
0
```

In [78]:
```
print(l)
print(l1)
```

```
[20, 50, 70, 20, 20]
[20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]
```

In [79]:
```
a = [1,2,3] #extend is used to extend the variable values using the other variables
b = [4,5,6]
a.extend(b)
```

In [80]:
```
a
```

Out[80]:
```
[1, 2, 3, 4, 5, 6]
```

In [81]:
```
l2 = []
l2
```

Out[81]:
```
[]
```

In [82]:
```
l2.extend(l)  #the values of l will be given to l2
l2
```

Out[82]:
```
[20, 50, 70, 20, 20]
```

In [85]:
```
l2.extend(l1)
l2
```

Out[85]:
```
[20,
 50,
 70,
 20,
 20,
 20,
 3.4,
 (3+2j),
 True,
 'hello',
 [1, 4, 5],
 20,
 3.4,
 (3+2j),
 True,
 'hello',
 [1, 4, 5]]
```

In [84]:
```
a = [1,2,3]
b = [4,5,6]

a.extend(b)
a
```

Out[84]:
```
[1, 2, 3, 4, 5, 6]
```

In [86]:
```
l[0]
```

Out[86]:
```
20
```

In [87]:
```
l[0] = 100 #list is mutable(changing the list value by its index position)
l
```

Out[87]:
```
[100, 50, 70, 20, 20]
```

In [89]:
```
l[1] = 500
l
```

Out[89]:
```
[100, 500, 70, 20, 20]
```

In [90]:
```
l.index(500)
```

Out[90]:
```
1
```

In [91]:
```
l.index(20) #it returns the first index position value but if we have duplicate val
```

Out[91]:
```
3
```

In [94]:
```
l.insert(2,33) #using insert we can insert values btween the values selected (index
l
```

Out[94]:
```
[100, 500, 33, 70, 20, 20, 20]
```

In [100…
```
l1
```

Out[100…
```
[20, 3.4, (3+2j), True, 'hello', [1, 4, 5]]
```

In [101…
```
l1.pop()  #it will remove last element
```

Out[101…
```
[1, 4, 5]
```

In [102…
```
l1
```

Out[102…
```
[20, 3.4, (3+2j), True, 'hello']
```

In [103…
```
l1.pop(3)  #if we give argument..it deletes the 3rd position value
```

Out[103…
```
True
```

In [104…
```
l1
```

Out[104…
```
[20, 3.4, (3+2j), 'hello']
```

In [117…
```
l2
```

Out[117…
```
[50,
 70,
 20,
 20,
 20,
 3.4,
 (3+2j),
 True,
 'hello',
 [1, 4, 5],
 20,
 3.4,
 (3+2j),
 True,
 'hello',
 [1, 4, 5]]
```

In [114…
```
l2.remove(20)
l2
```

Out[114…
```
[50,
 70,
 20,
 20,
 20,
 3.4,
 (3+2j),
 True,
 'hello',
 [1, 4, 5],
 20,
 3.4,
 (3+2j),
 True,
 'hello',
 [1, 4, 5]]
```

In [118…
```
l.reverse()  #it just mentions the reverse orderl
l
```

Out[118…
```
[20, 20, 70, 33, 500, 100]
```

In [120…
```
l.sort()
l
```

Out[120…
```
[20, 20, 33, 70, 100, 500]
```

In [121…
```
l.sort(reverse=True)  #it reverse funtion is added and is TRUE,the values are sorte
l
```

Out[121…
```
[500, 100, 70, 33, 20, 20]
```

In [122…
```
l.sort(reverse=False)  #asc order
l
```

Out[122…
```
[20, 20, 33, 70, 100, 500]
```

In [124…
```
l3= [10,20,30,40]
l3[2] = [13]
l3
```

Out[124…
```
[10, 20, [13], 40]
```

In [ ]: